

한충희

게임 클라이언트 프로그래머 (신입) · Unity · C#

010-2783-9414 · rislgowjra@gmail.com · 인천광역시 남동구

GitHub github.com/Huihans · 포트폴리오 huihans.github.io/Wooduduk.io

Unity·C#로 실시간 1v1 온라인 멀티와 확장 가능한 콘텐츠 시스템을 직접 설계·구현한 신입 클라이언트 프로그래머입니다. 기존 코어를 수정하지 않고 기능을 얻는 훅·이벤트 기반 설계와, 자동 검증으로 회귀를 미리 잡는 습관을 갖고 있습니다. 지속적으로 콘텐츠가 추가되는 라이브 서비스에 안정적으로 기여하는 데 강점이 있습니다.

보유 기술

- 언어 · C# (주력)
- 엔진 · Unity 6.3 (URP)
- 네트워크·실시간 · Firebase 서버리스(Realtime Database·Cloud Functions 연동), 실시간 1v1 룸 동기화(throttle write·구독)
- 설계·패턴
 - 훅 기반 확장 설계 — 기존 코어를 수정하지 않고 신규 콘텐츠를 얻는 구조
 - 옵저버(이벤트) 기반 관심사 분리 — 게임플레이 로직과 연출·UI를 단방향 이벤트로 격리
 - 데이터 구동 설계(ScriptableObject + 카탈로그/팩토리), 결정론 로직(시드 기반·입력 순서 무관), 인터페이스 경계 모듈화, SOLID
- 품질·검증 · EditMode 단위 테스트, 에디터 메뉴 기반 자동 검증 체계 직접 구축(테스트 프레임워크 없는 프로젝트에서 회귀 차단)
- 백엔드·데이터 · PlayFab(게임 BaaS) 연동, Google Apps Script 기반 데이터 파이프라인(Google Sheets 파싱·검증)
- AI 활용 개발 · AI 코딩 에이전트로 설계 검토·빌드 전 코드베이스 전체 스캐닝 — 컨벤션 위반·회귀를 사전 차단

프로젝트

우드득!(Wooduduk) — 하이브리드 캐주얼 스코어어택 게임

2026.06.03 ~ 진행 중 | 팀 5명(QA 전담 인원 없이 팀 자체 QA), 본인 = 클라이언트 시스템 파트 | PC·모바일 | Unity 6.3 (URP) / C#

슬롯을 굴려 나온 결과로 장작을 패는 스태프 방식 스코어어택 게임. 콤보·배율과 정산이 핵심.

- 확장형 카드 효과 시스템 (콘텐츠 10종 추가) — 슬롯 코어에 신규 효과 훅과 집계 전략(곱·합·최솟값·순차)을 정의하고, 카드 10종을 그 위에 얹었습니다. 기존 판정 엔진과 기존 카드 8종은 한 줄도 수정하지 않았고, 신규 파일도 만들지 않았습니다. 심볼 재라벨처럼 서로 충돌할 수 있는 효과는 원본 스냅샷만 읽도록 설계해 카드 간 적용 순서와 무관하게 동작하도록 만들었습니다.
- 자동 검증 체계 구축 — QA 전담 인원 없이 팀원이 직접 QA를 하는 환경이라, 수동 확인만으로는 회귀를 놓친다고 판단했습니다. 테스트 프레임워크가 도입되지 않은 프로젝트여서 에디터 메뉴로 실행하는 자체

검증 체계를 만들어 **카드 효과 48종 케이스를 자동 검증**하도록 했습니다. 중첩 구매 시 효과가 지수적으로 증폭되는 밸런스 리스크도 이 과정에서 발견해 팀에 리포트했습니다.

- **실시간 1v1 온라인 멀티 (클라이언트)** — Firebase 서버리스 기반 1v1 룸을 구현. 양측이 같은 시드로 결정론 시뮬을 돌리고, 각자의 라이브 상태(점수·콤보·체온·생존)를 throttle 방식(2~4Hz)으로 동기화해 상대 상태를 실시간 HUD로 표시. 전용 게임서버 없이 Cloud Functions 권위로 시드·랭킹을 처리하는 구조.
- **고스트풀 매칭 로직** — 티어·판수 기반 결정론적 버킷 배정과 시드 기반 균등 선택, 풀 부족 시 4단계 풀백(밴드 확장 → 티어 확장 → 필러 → 언더필)을 순수 로직으로 구현. 외부 백엔드를 **인터페이스로만 의존**해, 백엔드 완성 전에도 모듈 단독 빌드·EditMode 테스트가 가능하도록 설계(9종 테스트 케이스로 결정론 검증).
- **튜토리얼 UX 개편 (상황 인지형 유도)** — "무엇을 해야 할지 모르겠다"는 피드백을 받아, 상시 노출되는 안내 UI 대신 **필요한 순간에만 대상을 강조하고 행동하면 사라지는** 방식으로 재설계했습니다. 게임 상태를 읽어 다음 행동을 판정하는 부분을 실패와 무관한 순수 함수로 분리해 자동 검증이 가능하게 했고, 기존 조준 하이라이트와의 충돌은 억제 플래그로 조율했습니다.
- **기획자 협업 데이터 파이프라인 (Google Sheets → Apps Script → PlayFab)** — 기획자가 익숙한 Google Sheets에서 밸런스 데이터를 작성하면 Apps Script가 파싱·검증해 게임 백엔드로 반영하는 구조를 구현. 개발자 개입 없이 기획자가 직접 데이터를 수정·배포할 수 있게 해 이터레이션 속도를 높였습니다.

배운 점

- **"기존 코드를 고치지 않고 확장할 수 있는가"가 콘텐츠 추가 속도를 결정한다**는 것을 체감했습니다. 흑과 집계 전략을 먼저 정의해두니 카드 10종을 엮는 동안 기존 기능의 회귀가 한 건도 발생하지 않았습니다.
- 테스트 도구가 없는 환경에서도 검증 체계는 만들 수 있고, 그 체계가 **밸런스 결함까지 조기에 드러낸다**는 것을 배웠습니다.
- 외부 의존(백엔드)을 인터페이스로 격리해두니 백엔드 완성 전에도 내 모듈을 독립적으로 완성·검증할 수 있었습니다. "경계를 인터페이스로 긋는 설계"가 협업 속도와 안정성을 동시에 끌어올린다는 것을 체감했습니다.

교육

- **넥스트러너스 (게임 개발 부트캠프)** · 2025.12 ~ 2026.07 (7개월, 2026.07 수료 예정) — Unity 기반 게임 프로젝트 구현 및 팀 프로젝트 수행
- **운산공업고등학교 졸업** (2018.02)

기타

- 병역 · 만기전역